

第20回 JavaScript の基本 5

実習時のフォルダ構成

第 19 回と同じ形式で、Web_名前 の中に 20 フォルダを作る。20 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 20
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、すべて sample/main.js に書く）
- exercise：演習で作るファイル（演習 1～4、発展）

sample/index.html と exercise/index.html には、どちらも次の内容を書く（第 19 回の sample/index.html をコピーして、<title> だけ変えてもよい）。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本5</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

講義内容

以降の講義内容のコードは、sample/main.js に、上から順に書き足していく。書いたら sample/index.html をブラウザで開き、F12 キーの開発者ツール（コンソール）で結果を確認する。

1. 前回の復習

- for 文で、同じ処理を繰り返せる。配列[i] と組み合わせると、配列のすべての要素を順に処理できる。

```
const scores = [80, 45, 92];
for (let i = 0; i < scores.length; i++) {
  console.log(scores[i]);    // 80 45 92
}
```

- 今回は、処理にまとめて名前を付ける「関数」と、関係のある値をまとめる「オブジェクト」を学ぶ。

2. 関数

いくつかの処理をまとめて名前を付けたものを「関数」という。一度作っておけば、名前を書くだけで何回でも実行できる。

```
function 関数名() {  
    実行する処理  
}
```

```
function showTitle() {  
    console.log("=== SHOOTING GAME ===");  
}  
  
showTitle();  
showTitle();
```

- `function showTitle() { }` の部分は、関数を**作る**（定義する）だけ。この時点では、中の処理は実行されない。
- `showTitle();` のように、名前の後ろに `()` を付けて書くと、関数の中の処理が実行される（関数を**呼び出す**）。
- 今までに使ってきた `console.log()` や `Math.random()` も、JavaScript に最初から用意されている関数の仲間である。
- この授業では、関数は、呼び出すより前（ファイルの上の方）に作る。

3. 引数

関数に値を渡して、その値を使った処理をさせることができる。渡す値を「引数（ひきすう）」という。

```
function showDamage(damage) {  
    console.log(`${damage}のダメージ!`);  
}  
showDamage(10);           // 10のダメージ!  
showDamage(25);          // 25のダメージ!
```

- `showDamage(10);` と呼び出すと、`()` の中の `10` が、関数の `damage` に入ってから、中の処理が実行される。
- 引数は、`,` で区切って、いくつでも渡せる。渡す順番と、受け取る順番は同じになる。

```
function showStatus(name, hp) {  
    console.log(`${name}の残りHP: ${hp}`);  
}  
showStatus("スライム", 30);    // スライムの残りHP: 30
```

4. 戻り値

関数の中で計算した結果を、呼び出した場所へ返すことができる。返す値を「戻り値」といい、`return` で返す。

```
function calcDamage(attack, defense) {  
  return attack - defense;  
}  
const damage = calcDamage(40, 15);  
console.log(damage);           // 25
```

- `calcDamage(40, 15)` の部分が、戻り値の `25` に置き換わると考える。
- `return` が実行されると、関数はそこで終わる。

真偽値 (`true` / `false`) を返す関数は、`if` 文の条件に使える。

```
function isAlive(hp) {  
  return hp > 0;  
}  
console.log(isAlive(30));           // true  
console.log(isAlive(0));           // false  
if (!isAlive(0)) {  
  console.log("倒れた");  
}
```

- `hp > 0` の比較の結果 (`true` か `false`) が、そのまま戻り値になる。
- 真偽値を返す関数には、`isAlive` (生きているか) のように、`is` で始まる名前を付けることが多い。

5. 関数の外の変数を、関数の中で使う・書き換える

関数の外 (ファイルの上の方) で作った変数は、関数の中でも使え、書き換えることもできる。

```
let score = 0;  
function addScore(point) {  
  score += point;  
}  
addScore(100);  
addScore(50);  
console.log(score);           // 150
```

- 得点のように、ゲーム全体で使い続ける値は、関数の外で作る。
- 関数の中で作った変数 (引数も含む) は、その関数の中だけで使える。関数の外からは使えない。

6. オブジェクト

自機の位置 (x, y)、大きさ (幅, 高さ)、速さのように、関係のある値をまとめて 1 つにしたものを「オブジェクト」という。`{ }` の中に、`名前: 値` を、`,` で区切って並べる。

```
const player = {
  x: 100,
  y: 400,
  w: 32,
  h: 32,
  speed: 5
};
console.log(player.x);           // 100
console.log(player.speed);       // 5
player.x += player.speed;        // 右へ動かす
console.log(player.x);           // 105
```

名前（プロパティ）	値	意味
x	100	横の位置
y	400	縦の位置
w	32	幅（width）
h	32	高さ（height）
speed	5	速さ

- オブジェクトの中の1つ1つの値を「プロパティ」という。
- オブジェクト.プロパティ名 で取り出す（例：player.x）。変数と同じように、計算や代入ができる。
- 配列は「番号」で取り出し、オブジェクトは「名前」で取り出す。

7. const と、配列・オブジェクトの中の値

第19回では、要素を書き換える配列に `let` を使った。実は、`const` で作った配列やオブジェクトでも、中の値は書き換えられる。

```
// player = { x: 0, y: 0 }; // エラー：const の変数には、別の値を代入し直せない
player.y = 380;           // const のオブジェクトでも、中の値は書き換えられる
console.log(player.y);    // 380

const enemyHps = [30, 50, 80];
enemyHps[1] = 20;         // const の配列でも、要素は書き換えられる
console.log(enemyHps);    // [30, 20, 80]
```

- `const` で禁止されるのは、「変数そのものに、別の値を代入し直すこと」（`player = ...`）。
- 中のプロパティや要素を書き換えること（`player.y = ...`、`enemyHps[1] = ...`）は、`const` でもできる。
- この授業では、ここから次のルールにする。
- 変数そのものに、別の値を代入し直すことがある → `let`
- 代入し直さない（中の値を書き換えるだけの場合も含む） → `const`

8. 関数とオブジェクトを組み合わせる

自機のオブジェクトを、関数で動かしたり、表示したりする。

```
const screenWidth = 400;

function updatePlayer() {
  player.x += player.speed;
  // 画面の端で止める
  if (player.x + player.w > screenWidth) {
    player.x = screenWidth - player.w;
  }
}

function showPlayer() {
  console.log(`自機の位置 : x = ${player.x}, y = ${player.y}`);
}

for (let i = 0; i < 3; i++) {
  updatePlayer();
  showPlayer();           // x = 110 → 115 → 120
}
```

- 「動かす」処理を `updatePlayer`、「表示する」処理を `showPlayer` のように、役割ごとに関数に分けると、どこで何をしているかが分かりやすくなる。
- ゲームでは、「動かす（更新）」と「描く（描画）」を分けて、何回も繰り返す。第 22 回で、この形を使ってゲームを動かす。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて、F12 の開発者ツール（コンソール）で確認する。

演習 1 関数を作って呼び出す

- 「GAME OVER」と表示する関数 `showGameOver` を作り、2 回呼び出す。
- 得点を引数で受け取り、「得点：〇〇点」と表示する関数 `showScore` を作って、呼び出す。

演習 2 戻り値のある関数

- 倒した敵の数を引数で受け取り、「敵の数 × 100」を戻り値として返す関数 `calcScore` を作る。5 を渡して、戻り値を表示する。
- x 座標を引数で受け取り、「0 以上 かつ 400 以下」なら `true`、そうでなければ `false` を返す関数 `isInScreen` を作る。
- `isInScreen` を `if` 文の条件に使い、画面の外ときに「画面の外」と表示する。

演習 3 オブジェクトを作る

- 敵を表すオブジェクト `enemy` を、`const` で作る。プロパティは、名前 (`name`)、HP (`hp`)、x 座標 (`x`)、y 座標 (`y`) の 4 つ。
- 「○○のHP：△△」と表示する。
- HP を 20 減らしてから、もう一度表示する。

演習 4 関数とオブジェクトを組み合わせる

- 得点を入れる変数を、関数の外に `let` で作る（最初は 0）。
- ダメージを引数で受け取る関数 `damageEnemy` を作る。関数の中で、次を行う。
- 演習 3 の `enemy` の HP から、ダメージを引く。
- HP が 0 以下なら「○○を倒した！」と表示し、得点に 100 を足す。そうでなければ「○○の残り HP：△△」と表示する。
- `damageEnemy` を 3 回呼び出して、敵を倒す。最後に得点を表示する。

発展（任意）

- オブジェクトを引数で受け取り、その `name`・`x`・`y` を表示する関数 `showCharacter` を作る。演習 3 の `enemy` を渡して呼び出す。
- 値・最小値・最大値の 3 つを引数で受け取り、「最小値以上 かつ 最大値以下」なら `true` を返す関数 `isInRange` を作る。`enemy.x` や `enemy.y` を渡して試す。